In [15]:
```
# ==========================================================
# PROJECT: Data Cleaning & Preparation - Bike Sharing Dataset
# AUTHOR: Data Analyst Portfolio
# TOOL: Python (Pandas, NumPy, Scikit-Learn)
# OBJECTIVE: Prepare raw bike sharing data for analysis/modeling
# ==========================================================
```

In [16]:
```
pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in C:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-package
s (1.8.0)
Requirement already satisfied: numpy>=1.24.1 in C:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-packag
es (from scikit-learn) (2.4.4)
Requirement already satisfied: scipy>=1.10.0 in C:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-packag
es (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.3.0 in C:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-packag
es (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in C:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site
-packages (from scikit-learn) (3.6.0)
Note: you may need to restart the kernel to use updated packages.
```

In [17]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
```

In [18]:
```
# 1. IMPORT DATA

df_raw = pd.read_csv("bikes_sharing.csv")

print("=== Dataset Structure ===")
print(df_raw.info())

print("\n=== First 10 Observations ===")
print(df_raw.head(10))

print("\n=== Summary Statistics ===")
print(df_raw.describe(include='all'))
```

```
=== Dataset Structure ===
<class 'pandas.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   datetime    10886 non-null  str
 1   season      10886 non-null  int64
 2   holiday     10886 non-null  int64
 3   workingday  10886 non-null  int64
 4   weather     10886 non-null  int64
 5   temp        10886 non-null  float64
 6   atemp       10886 non-null  float64
 7   humidity    10886 non-null  int64
 8   windspeed   10886 non-null  float64
 9   casual      10886 non-null  int64
 10  registered  10886 non-null  int64
 11  count       10886 non-null  int64
dtypes: float64(3), int64(8), str(1)
memory usage: 1020.7 KB
None

=== First 10 Observations ===
     datetime  season  holiday  workingday  weather   temp   atemp  humidity  \
0  2011-01-01       1        0           0        1   9.84  14.395        81
1  2011-01-01       1        0           0        1   9.02  13.635        80
2  2011-01-01       1        0           0        1   9.02  13.635        80
3  2011-01-01       1        0           0        1   9.84  14.395        75
4  2011-01-01       1        0           0        1   9.84  14.395        75
5  2011-01-01       1        0           0        2   9.84  12.880        75
6  2011-01-01       1        0           0        1   9.02  13.635        80
7  2011-01-01       1        0           0        1   8.20  12.880        86
8  2011-01-01       1        0           0        1   9.84  14.395        75
9  2011-01-01       1        0           0        1  13.12  17.425        76

   windspeed  casual  registered  count
0     0.0000       3          13     16
1     0.0000       8          32     40
2     0.0000       5          27     32
3     0.0000       3          10     13
4     0.0000       0           1      1
5     6.0032       0           1      1
```

```
6     0.0000       2           0      2
7     0.0000       1           2      3
8     0.0000       1           7      8
9     0.0000       8           6     14

=== Summary Statistics ===
          datetime        season       holiday    workingday       weather  \
count        10886  10886.000000  10886.000000  10886.000000  10886.000000
unique         456           NaN           NaN           NaN           NaN
top     2011-01-01           NaN           NaN           NaN           NaN
freq            24           NaN           NaN           NaN           NaN
mean           NaN      2.506614      0.028569      0.680875      1.418427
std            NaN      1.116174      0.166599      0.466159      0.633839
min            NaN      1.000000      0.000000      0.000000      1.000000
25%            NaN      2.000000      0.000000      0.000000      1.000000
50%            NaN      3.000000      0.000000      1.000000      1.000000
75%            NaN      4.000000      0.000000      1.000000      2.000000
max            NaN      4.000000      1.000000      1.000000      4.000000

               temp         atemp      humidity     windspeed        casual  \
count   10886.00000  10886.000000  10886.000000  10886.000000  10886.000000
unique          NaN           NaN           NaN           NaN           NaN
top             NaN           NaN           NaN           NaN           NaN
freq            NaN           NaN           NaN           NaN           NaN
mean       20.23086     23.655084     61.886460     12.799395     36.021955
std         7.79159      8.474601     19.245033      8.164537     49.960477
min         0.82000      0.760000      0.000000      0.000000      0.000000
25%        13.94000     16.665000     47.000000      7.001500      4.000000
50%        20.50000     24.240000     62.000000     12.998000     17.000000
75%        26.24000     31.060000     77.000000     16.997900     49.000000
max        41.00000     45.455000    100.000000     56.996900    367.000000

          registered         count
count   10886.000000  10886.000000
unique           NaN           NaN
top              NaN           NaN
freq             NaN           NaN
mean      155.552177    191.574132
std       151.039033    181.144454
min         0.000000      1.000000
25%        36.000000     42.000000
50%       118.000000    145.000000
```

```
75%       222.000000    284.000000
max       886.000000    977.000000
```

In [19]:
```python
# 2. MISSING VALUES CHECK

print("\n=== Missing Values per Column ===")
print(df_raw.isna().sum())
```

```
=== Missing Values per Column ===
datetime      0
season        0
holiday       0
workingday    0
weather       0
temp          0
atemp         0
humidity      0
windspeed     0
casual        0
registered    0
count         0
dtype: int64
```

In [20]:
```python
# 3. REMOVE DUPLICATES

df_nodup = df_raw.drop_duplicates()
print(f"\nDuplicates removed: {df_raw.shape[0] - df_nodup.shape[0]}")
```

```
Duplicates removed: 15
```

In [21]:
```python
# 4. IMPUTATION (MEDIANS)

numeric_cols = ["temp", "atemp", "humidity", "windspeed"]

medians = df_nodup[numeric_cols].median()

df_clean = df_nodup.copy()
df_clean[numeric_cols] = df_clean[numeric_cols].fillna(medians)

print("\n=== Medians Used for Imputation ===")
print(medians)
```

```
=== Medians Used for Imputation ===
temp         20.500
atemp        24.240
humidity     62.000
windspeed    12.998
dtype: float64
```

In [22]:

```python
# 5. OUTLIER REMOVAL (IQR METHOD)

Q1 = df_clean["count"].quantile(0.25)
Q3 = df_clean["count"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df_no_outliers = df_clean[(df_clean["count"] >= lower) & (df_clean["count"] <= upper)]

print("\n=== Outlier Thresholds ===")
print(f"Lower: {lower}, Upper: {upper}")
print(f"Rows removed: {df_clean.shape[0] - df_no_outliers.shape[0]}")
```

```
=== Outlier Thresholds ===
Lower: -318.5, Upper: 645.5
Rows removed: 307
```

In [23]:

```python
# 6. FEATURE ENGINEERING

df_final = df_no_outliers.copy()

# Temperature categories
df_final["temp_level"] = pd.cut(
    df_final["temp"],
    bins=[-np.inf, 10, 25, np.inf],
    labels=["Cold", "Mild", "Hot"]
)

# Demand level
df_final["demand_level"] = pd.cut(
    df_final["count"],
    bins=[-np.inf, 100, 500, np.inf],
    labels=["Low", "Medium", "High"]
```

```
)

# Humidity risk
df_final["humidity_risk"] = df_final["humidity"] * df_final["windspeed"]

# High demand binary
df_final["high_demand"] = (df_final["count"] > 200).astype(int)

# Convert datetime
df_final["datetime"] = pd.to_datetime(df_final["datetime"])

# Time-based features
df_final["hour"] = df_final["datetime"].dt.hour
df_final["day"] = df_final["datetime"].dt.day
df_final["month"] = df_final["datetime"].dt.month
df_final["is_weekend"] = df_final["datetime"].dt.weekday.isin([5, 6]).astype(int)

print("\n=== Feature Engineering Completed ===")
print(df_final.head(10))
```

```
=== Feature Engineering Completed ===
    datetime  season  holiday  workingday  weather   temp   atemp  humidity  \
0 2011-01-01       1        0           0        1   9.84  14.395        81
1 2011-01-01       1        0           0        1   9.02  13.635        80
2 2011-01-01       1        0           0        1   9.02  13.635        80
3 2011-01-01       1        0           0        1   9.84  14.395        75
4 2011-01-01       1        0           0        1   9.84  14.395        75
5 2011-01-01       1        0           0        2   9.84  12.880        75
6 2011-01-01       1        0           0        1   9.02  13.635        80
7 2011-01-01       1        0           0        1   8.20  12.880        86
8 2011-01-01       1        0           0        1   9.84  14.395        75
9 2011-01-01       1        0           0        1  13.12  17.425        76

   windspeed  casual  registered  count temp_level demand_level  \
0     0.0000       3          13     16       Cold          Low
1     0.0000       8          32     40       Cold          Low
2     0.0000       5          27     32       Cold          Low
3     0.0000       3          10     13       Cold          Low
4     0.0000       0           1      1       Cold          Low
5     6.0032       0           1      1       Cold          Low
6     0.0000       2           0      2       Cold          Low
7     0.0000       1           2      3       Cold          Low
8     0.0000       1           7      8       Cold          Low
9     0.0000       8           6     14       Mild          Low

   humidity_risk  high_demand  hour  day  month  is_weekend
0           0.00            0     0    1      1           1
1           0.00            0     0    1      1           1
2           0.00            0     0    1      1           1
3           0.00            0     0    1      1           1
4           0.00            0     0    1      1           1
5         450.24            0     0    1      1           1
6           0.00            0     0    1      1           1
7           0.00            0     0    1      1           1
8           0.00            0     0    1      1           1
9           0.00            0     0    1      1           1
```

In [24]:

```python
# 7. NORMALIZATION (STANDARDIZATION)

scaler = StandardScaler()
cols_to_scale = ["temp", "atemp", "humidity", "windspeed", "count"]
```

```
df_final_scaled = df_final.copy()
df_final_scaled[cols_to_scale] = scaler.fit_transform(df_final[cols_to_scale])

print("\n=== Standardized Variables Summary ===")
print(df_final_scaled[cols_to_scale].describe())
```

```
=== Standardized Variables Summary ===
               temp         atemp      humidity     windspeed         count
count  1.056400e+04  1.056400e+04  1.056400e+04  10564.000000  1.056400e+04
mean  -2.152345e-16 -1.506641e-16 -1.183789e-16      0.000000  6.457034e-17
std    1.000047e+00  1.000047e+00  1.000047e+00      1.000047  1.000047e+00
min   -2.475109e+00 -2.684902e+00 -3.233193e+00     -1.563952 -1.120203e+00
25%   -7.880867e-01 -8.056534e-01 -7.886736e-01     -0.707297 -8.700581e-01
50%    5.542427e-02  8.936760e-02 -8.507678e-03      0.026393 -2.350752e-01
75%    7.934963e-01  8.951819e-01  8.236693e-01      0.515792  6.051547e-01
max    2.691396e+00  2.596017e+00  1.967913e+00      5.409791  3.010393e+00
```

In [25]:

```
# 8. EXPORT CLEAN DATASET

df_final_scaled.to_csv("bikes_clean_python.csv", index=False)
print("\n=== Export Completed: bikes_clean_python.csv ===")
```

```
=== Export Completed: bikes_clean_python.csv ===
```

In [ ]: